

Should we import function bodies to get the better optimizations?

Document #: P3033R0
Date: 2023-11-01
Project: Programming Language C++
Audience: SG15
Reply-to: Chuanqi Xu
<chuanqi.xcq@alibaba-inc.com>

Contents

- [1 Abstract](#)
- [2 Background and Motivation](#)
- [3 Proposed Solutions](#)
- [4 Practical impact for recompilations](#)
- [5 Practical impact for build systems](#)
- [6 Practical impact for compilers](#)
- [7 Practical impact for runtime performances](#)
- [8 A discarded idea in clang](#)
- [9 Acknowledgments](#)

§ 1 Abstract

This paper discusses whether or not should we import function bodies to the optimizer to get the better optimizations. Then we will discuss the practical impact with the decision we made for build systems, compilers and runtime performances.

§2 Background and Motivation

The motivation of the paper comes from a clang issue report: <https://github.com/llvm/llvm-project/issues/60996>. The issue said, the compilation time grows non linearly with optimizations. The secret here is that clang will import function bodies to optimizers to get more optimization opportunities. For example,

```
// a.cppm
export module a;
export int a() { return 43; }

// use.cpp
import a;
int use() { return a(); }
```

With optimizations enabled, the generated code for use() in clang will be like:

```
int use() { return 43; }
```

It looks pretty reasonable, right? It also matches the zero abstract principle in my mind. However, the cost is that when we compile use.cpp, the optimizer will work on a() too. We can image this can be a big cost when scaling up.

This is the problem and this is the tradeoffs we need to face. So far, I thought it is basically a matter of implementation details. But Daniel pointed out that this is related to ABI dependencies and potential ODR violations and he suggests to send a paper to SG15 to get a consensus in the vendors.

The story is, reusing the above example, after we build the project once, the build dir may look like:

```
a.o a.pcm use.o libuse.so
```

And now we change the implementation of `a()` to return 44; and recompile the project, what's going on? There are 2 possible compiling results:

```
rebuild a.o  
rebuild a.pcm  
rebuild use.o  
link libuse.so
```

or

```
rebuild a.o  
rebuild a.pcm  
link libuse.so
```

The difference here is whether or not should we recompile `use.cpp` to `use.o`. If we don't import the function body of `a()` to `use.cpp`, the generated code for `use()` will simply be a call to an external function `a()` all the way. But if we import the function body of `a()` to `use.cpp`, the function body of `a()` will become part of the body for `use()`. Then the ABI dependency information get changed.

§3 Proposed Solutions

When we change a non-inline function body in a module purview, the BMI contents for the corresponding module unit shouldn't change.

§4 **Practical impact for recompilations**

When the users only touches a body of a non-inline function in the purview of a module interface unit and recompile the project, only the touched module interface unit needs to be recompiled. All other compilations shouldn't happen. Here we excludes the linking the stage.

This should be able to improve the user experience significantly and this is the main motivation of the paper.

§5 **Practical impact for build systems**

If the build systems implement the semantics correctly to make source files dependent on BMI instead of the module units, there shouldn't be an impact to build systems.

For time-stamp based build system, it may be easy to implement the compare-and-swap mechanism. That said, generate the BMI to a temporary position and compare its content with the existing one and only replace the existing one when we find it is not the same. Or the build system can require the compilers to do this.

§6 Practical impact for compilers

According to Iain and Cameron, both GCC and MSVC don't write the non-inline function body to the BMI. So both of GCC and MSVC won't be affected. Following of the section we are discussing clang.

For clang to achieve this, only not importing non-inline function bodies is not enough. Clang need to remove the non-inline function bodies from the BMI. It implies that clang can't continue the current 2 phase compilation model.

The 2 phase compilation model is: when clang compiles a module interface unit, the unit will be compiled to the BMI at first, then the BMI will be compiled to the object file later. The advantage of the 2 phase compilation model is the potential higher parallelism. That said, we can start compile the waiting units after the BMI is generated. This is impossible for the one phase compilation model until we implement the server-client model actually. But the requirement of the 2 phase compilation is that the BMI should contain all the information needed to generate the object file. So if we don't want to change the BMI after we touch a body of non-inline function, we can't generate that body to the BMI. Then we loses the basis of 2 phase compilation.

§7 Practical impact for runtime performances

As we pointed out in the beginning of the paper, not importing function bodies may lose optimization opportunities. And the runtime performance is a key factor of C++ programs. This is the reason why I was trying to keep clang's behavior.

But David's words inspires me. He said, when we talk about the performance regression, we need a baseline. In practice, we are mainly rewriting/wrapping a header based library to a modular one. And the functions in headers are almost inline functions. That said, we may not get a performance regression after we refactor a header based library to a modular library even after we make this decision.

So in some degree, our decision doesn't introduce performance regressions.

Also it may be worth to mention that, the full LTO should be able to solve the issue since it can collect all the information in the project. The reason why I didn't treat LTO as the silver bullet since it is pretty expensive that people don't use it in daily jobs. But sure, people can still use the LTO to get the best runtime performance just like the time modules are not introduced.

§ 8 A discarded idea in clang

Previously, I had an idea to reduce the compilation time and save optimization opportunities is to wrap the optimized IR in the BMI. And we can directly import these optimized IR to its users. These optimized IRs are marked to not be optimized and they can only be inlined. Then in this way, the duplicated optimizations are avoided and we saved the optimization opportunities too.

But the idea should be discarded if we prefer the proposal. Since it still introduces the ABI dependencies to the users of modules.

§ 9 Acknowledgments

Many thanks to Daniel Ruoso, Iain Sandoe, David Blaikie, Cameron DaCamara, Ben Boeckel and David Stone for discussing the problems.